Tutorial #4-Lunar Buggy

Introduction

Welcome to the fourth in series of AppGameKit Studio tutorials. These tutorials are aimed at those of you who are just starting to explore the world of tier 1 (BASIC) coding. As you'll recall, we previously looked at the games of Pong, Reversi and Tanks.

From those games we learnt to control sprites, receive and respond to user input and gain a simple understanding of some basic AI.

This tutorial will expand once again on the coding techniques we covered in the previous three tutorials, and as before, everything new you learn here, will be of use in your future projects. You'll be guided through each of the steps clearly, and be encouraged to change various values and lines of code so that you can learn first-hand how simple changes can affect the code.

The entry level for these tutorials expects some initial understanding of the students in the use of AppGameKit Studio. For more beginner level tutorials they can read the AppGameKit Studio Online Guides and watch the YouTube Videos.

Students are also encouraged to join the Official AppGameKit Studio forums. There's a newbie section where new users will be helped by many experienced and friendly developers who use AppGameKit Studio all the time.

Objectives

These tutorials aim to teach coding and game development using a combination of examples and action based teaching. You will learn from studying the code snippets, reading and understanding the comments and gain an additional understanding of each section through a number of simple exercises aimed at reinforcing each lesson and will help build your confidence by demonstrating the ease with which the code can be altered.

This fourth tutorial aims to expand on the concepts of basic coding covered by the previous tutorials and is designed to lay the groundwork of future developments. During this tutorial we will:

- Use several variable types, all of which have been covered in the previous tutorials, re-enforcing the previous lessons
- Expand on techniques learnt in the previous tutorials and learn how to combine coding practices from them for different results
- Learn about simple sprite animation
- Start to explore 2D physics to create simple physics based effects
- Learn how to break your code up into easy to read files

At any point, examples of almost all coding commands can be accessed from the IDE (integrated development environment) simply by either clicking directly to the left of the command and pressing F1 on the keyboard. You are encouraged to learn this method from the beginning as it will prove to be an invaluable reference guide and is a resource often used even by coders who are very familiar with AppGameKit Studio.

You can also access a full command reference from the IDE. By clicking Help on the top tool bar, you can gain access to a number of reference sources, including video tutorials, a list of useful keyboard shortcuts as well as the command reference.

Each section of code is available as a separate project which can be simply loaded into the IDE, but it is recommended that you learn to type the code into the IDE, as this will help to develop a familiarity with AppGameKit Studio overall and will promote confidence in using the functions of the IDE as well as reinforcing the learning process.

Let's talk about #include and adding files

When your code gets large, many programmers find it useful to break their code into smaller groups called files. These are best thought of as seperate programs that are compiled as one single program, but are easy to inspect due to there compact size. In many cases an #include file won't contain more that 3 or 4 Functions, and ideally they will relate to a single part of your code. e.g. Code related to the player, code used by an AI etc.

You can create a new **#include** file by simply clicking on the File menu in the IDE, clicking New File and saving your files as **yourname.agc**, obviously, you'll replace **yourname** with whatever you want to call the file, and you must include the **.agc** in the file name otherwise it won't be recognised.

There is no limit to the number of #include files you can use, but each must be unique within your project. For you main code (main.agc) to access these files, you must add them to your main code (main.agc) at the start or they won't be included and compiled.

Open up the **main.agc** code now and take a look at these lines:

```
//this is where we link to our other files
//this one contains all of our player code, such as displaying the player, moving them,
//shooting and controlling bullets, etc
#include "playerstuff.agc"
//next we will link to everything terrain related, including building and moving the terrain
#include "terrainstuff.agc"
//now we'll link to all the functions that control alien things, moving, shooting etc,
#include "alienstuff.agc"
//the next set of functions we'll link to is all explosions, player and aliens
#include "explosionstuff.agc"
//the last set to link to controls what we'll call admin, this include the main menu,
updating scores, etc.
#include "adminstuff.agc"
```

Simply by adding **#include "abcd.agc"** you will be telling your code to compile that file along with the main code.

What game are we making?

For this tutorial, we are aiming to combine lots of the techniques we learnt in our previous tutorials and will combine many of these lesson to create a more rounded game. This time, we are making a game called Lunar Buggy, where the player fights their way across and infinate lunar landscape, leaping over moon rocks and craters, while battling an unknown alien menace.

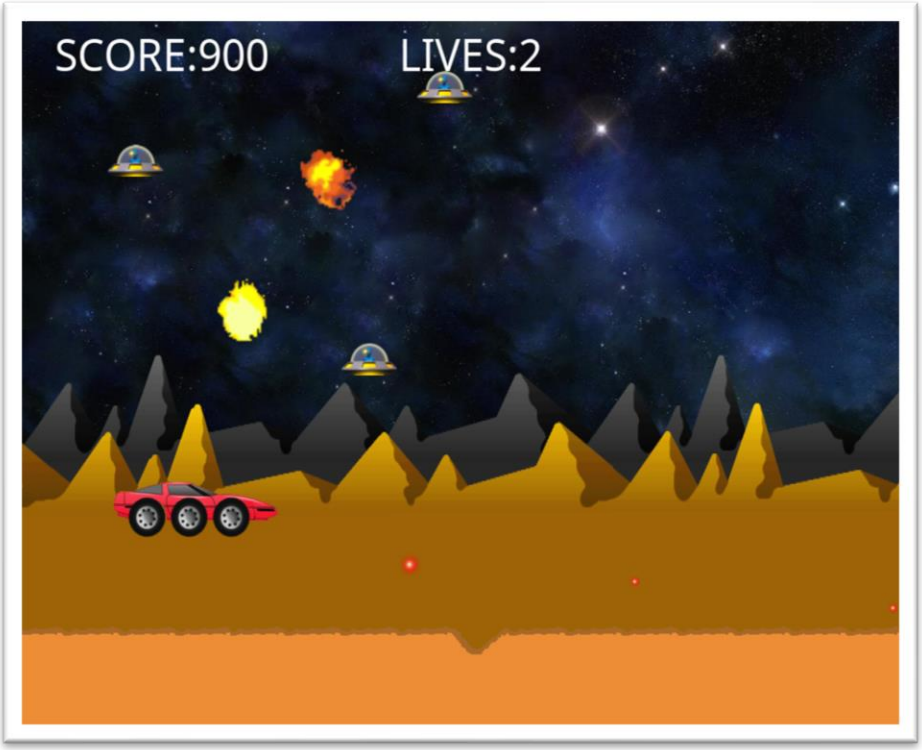

Fig 1 – The full game of Lunar Buggy

Overall, we will be covering the coding practices we've already covered in less detail than before, although the code is fully documented, and as always, we encourage you to read it carefully so you're sure you understand what is happening.

We'll spend more time on the new technique, and will of course expand on these at a later date.

Let's take a look at each #include file, and briefly refresh our memories of old practices as well as highlighting the new ones.

Main.agc

The code inside Main.agc does a couple of things, and can be thought of as the control code. It creates and assigns variables, and runs the main game loop. We'll just rely on the comments for older code, but we will go into more detail for new or lesser used commands. As we've covered the setting up of varibles several times before we won't repeat that code here, but you are encouraged to review the code itself to refresh your memory.

We will however take a closer look at the main DO – LOOP

```
do

        //if our background music has stopped playing
        //start it up again

        If GetMusicPlayingOgg(music)=0
                PlayMusicOgg(music)
        EndIf
```

The code above is of interest and is worthy of a little more explanation. Ogg music files are a very popular and useful type of sound file, ideally suited for music. You can adjust the volume of Ogg files, but they aren't really suited for sound effects, where the popular .Wav format is more commonly used. We will cover these differences in more detail in a future tutorial, but for now it's worth noting the suitablity of the two sound formats.

Let's carry on with the rest of the code.

```
    //set frame adjust to 60 * by the time it took to draw the last frame
    //this will produce a value we can multiple all movement by to ensure
    //the appearance of smooth movement
    frameadjust=60*GetFrameTime()
    //we store the current time as a performance saver
    //calling GetMilliseconds() using quite a lot of processing power
    //and as we rarely need 100% millisecond accuracy, a single call a frame
    //is enough
    currenttime=GetMilliseconds()

    //if the player isn't dead,
     //let's play the game
    if player.dead=0
    //update the terrain
    drawterrain()
     //get some input from the player
     //and handle it if required
     controlplayer()
      //move the player around
     moveplayer()
     //display the player by drawing the buggy and wheels again
      displayplayer()
    //move the terrain
     moveterrain()

     //we'll spawn the alien ships sparingly
    //however we could change this number
    //to have more or less ships, the higher the number, the fewer ships
    If random(1,1000)>990
    spawnalienship()
    Endif

Endif

//if the player is dead
If player.dead<>0
    //and has been dead for more than 4 seconds (or 4000 milliseconds)
    If currenttime-player.deadtime>4000
            //hide the buggy parts and all the wheels
            For move=150 to 157
                    SetSpritePosition(move,-10000,-10000)
```

```
                //and turn of the physics for now
                SetSpritePhysicsOff(move)
            Next move
            //make the player alive again
            //reset the buggy position
            player.dead=0
            player.x=300
            player.y=600
            //and reset the level
            //part of adminstuff.agc
            resetlevel()

            //if the player has run out of lives.
            //we'll just go back to the main menu
            if player.lives=0
            //part of adminstuff.agc
            mainmenu()
            Endif
        Endif
EndIf

    //regardless of wether the player is alive or dead
    //we'll need to keep moving the player bullets
    //part of playerstuff.agc
    movefire()
    //move the alien ships
    //part of alienstuff.agc
    movealienships()
    //move the alien bullets
    //part of alienstuff.agc
    movealienfire()
    //check on our explosions
    //part of explosionstuff.agc
    controlbangs()
    // and display our score and lives.
    // part of adminstuff.agc
    stats()
    //draw everything from the backbuffer to the frontbuffer (the screen)
    Sync()
    //and do it all again
loop
```

If you've been paying attention, the reasons for all of the above code should be obvious, if not, don't be worried about reading back through the previous tutorials.

EXERCISE: *Run the code and note how many alien ships appear. Now read through the code and see if you can work out how you can make more alein ships appear, and then less. Once done, reset the code to the original chance of spawning a new alien.*

OK, now let's look at our player control code. Open up the file, **playerstuff.agc**. We have covered all of the techniques here before, so for this section, we suggest you simply read through the code and make sure you understand what's happening.

```
        Function displayplayer()

            //set the player position
            //you'll recall the command RandomSign(1) from our pong tutorial and if you remember,
            //this takes the number in the () and has a 50 - 50 chance of making it negative.
            SetSpritePosition(player.sprite,player.x+RandomSign(1),player.y+RandomSign(1))

            //set the start position of the wheels with a slight random offset.
            wheelposx=player.x+16+RandomSign(1)

            //we place the wheels at a slight offset from the Lunar Buggy
            //adding a little randomness to make the buggy appear to rattle around.
            For placewheels=player.wheel1 to player.wheel3
                SetSpriteAngle(placewheels,player.wheelangle)
                SetSpritePosition(placewheels,wheelposx,player.y+(16+RandomSign(1)))
                //move the next wheel up
                inc wheelposx,48
                //finish the loop
            Next placewheels
```

```
        //inc the wheel angle,
        //when displaying the wheel we will rotate it to this angle
        //making it appear that the wheels are rotating
        inc player.wheelangle,4

        //we also check to see if the wheels have hit an obstacle
        //and obstacle in this case is a piece of terrain that isn't flat ground
        //and is therefore a rock or a crater.
        bang=0

        //loop though all 50 possible terrain sprites.
        For collision=1 to 50
                //if the terrain is being used (on screen in this case) and isn't a ground
                //terrain type, check for a collision.
                If terrain[collision].active=1 and terrain[collision].image<>ground

                        //check all 3 wheels
                        For checkwheels=player.wheel1 to player.wheel3
                        //see if the sprites collide,
                          If GetSpriteCollision(checkwheels,terrain[collision].sprite)
                                //flag that a collision has happened
                                bang=1
                                //jump out of both loops as we don't
                                //need to check anymore.
                                checkwheels=player.wheel3+1
                                collision=51
                              EndIf
                              //check next wheel
                        Next checkwheels
                EndIf
                //check next terrain
        Next collision

        //if we've had a collision
        //and the player isn't dead
        If bang=1 and player.dead=0
                //explode the buggy
                explodebuggy()
        endif

EndFunction

Function controlplayer()

        //backwards
        //A or left arrow
        If (GetRawKeyState(65) or GetRawKeyState(37)) and player.x>64
                dec player.x,6*frameadjust
        EndIf

        //forwards
        //D or right arrow
        If (GetRawKeyState(68) or GetRawKeyState(39)) and player.x<800
                inc player.x,6*frameadjust
        EndIf

        //jump
        //enter or up arrow
        If (GetRawKeyState(13) or GetRawKeyState(38)) and player.jumping=0
                player.jumping=1
                player.jumpdirection=-2
        Endif

        //fire
        //space or ctrl
        If (GetRawKeyState(32) or GetRawKeyState(17)) and currenttime-playerlastfire>500
                playerlastfire=currenttime
                playsound(playerzap)
                makeplayerfire()
        EndIf



EndFunction

Function moveplayer()

        //is the player jumping?
        If player.jumping<>0
```

```
                //move the buggy in the direction of the jump
                player.y=player.y+player.jumpdirection*frameadjust
                //don't let the buggy go too far up the screen
                //if the buggy is lower that postion 480 on the screen make the
                //buggy fall
                If player.y<480
                        player.jumpdirection=2
                Endif

                //if the buggy is higher than position 600, which is where the terrain is
                //stop it jumping as we know it's landed.
                //we also force it's position to be 600 to stop it falling lower
                //than the terrain
                If player.y>=600
                        player.y=600
                        player.jumping=0
                EndIf
        Endif

EndFunction

Function makeplayerfire()

        //find a bullet that's not being used.
        spare=0

        //we can fire forward and upwards,
        //so we use a loop to create 2 bullets
        //one for each direction
        For loopit=1 to 2

        For search=1 to 40
                if pfire[search].active=0
                spare=search
                search=41
                Endif
        Next search

        //if we are in the first loop
        //create the up bullet
        if loopit=1
                pfire[spare].active=1
                pfire[spare].x=player.x+36
                pfire[spare].y=player.y+12
                pfire[spare].direction=1
                //if we're not in the first loop, we must be in the second loop
                //so create the forward bullet.
                else
                pfire[spare].active=1
                pfire[spare].x=player.x+96
                pfire[spare].y=player.y+24
                pfire[spare].direction=2
        Endif

        //Set the sprite position
        SetSpritePosition(pfire[spare].sprite,pfire[spare].x,pfire[spare].y)

Next loopit

EndFunction

Function movefire()

        //now we've created our bullets, let's move them
        //loop through them

        For movefire=1 to 40
                //is it active?
                If pfire[movefire].active=1

                //direction 1 means the bullet is moving upwards
                If pfire[movefire].direction=1
                        dec pfire[movefire].y,6*frameadjust
                EndIf

                //direction 2 means the bullet is moving forward
                If pfire[movefire].direction=2
                        inc pfire[movefire].x,6*frameadjust
                EndIf
```

```
                //update the screen position
                SetSpritePosition(pfire[movefire].sprite,pfire[movefire].x,pfire[movefire].y)

                //call a special routine to see if our bullet has hit some terrain
                checkterrainhit(movefire)

                //we also want to know if it's left the screen
                //if so, make it inactive, so we can use it again
                if pfire[movefire].x>1080 or pfire[movefire].y<-80
                        pfire[movefire].active=0
                Endif

                //now we check each alien to see if we've shot it down
                For isalienhit=1 to 20
                        //but only if it's active
                        If aliens[isalienhit].active<>0
                                //have the sprites collided
                                if
GetSpriteCollision(aliens[isalienhit].sprite,pfire[movefire].sprite)
                                //if so, make the alien inactive
                                aliens[isalienhit].active=0
                                //call the special makebang function
                                //this will generate 3 different types of explosion
                                //1 if the bullet hits the ground
                                //2 if it hits a rock
                                //3 if it hits a ship
                                makebang(aliens[isalienhit].x,aliens[isalienhit].y,3)
                                //increase the player score
                                inc player.score,100
                                //move the alien offscreen
                                aliens[isalienhit].x=-10000
                                aliens[isalienhit].y=-10000

        SetSpritePosition(aliens[isalienhit].sprite,aliens[isalienhit].x,aliens[isalienhit].y
)
                                //make the bullet inactive and hide the bullet offscreen
                                pfire[movefire].active=0
                                SetSpritePosition(pfire[movefire].sprite,-10000,-10000)
                                //jump out of the loop
                                isalienhit=21
                                EndIf
                        EndIf
                        //carry on with the loop if needed
                        Next isalienhit

                Endif
                //check the next bullet
        Next movefire

EndFunction
```

EXERCISE: *Run the code and note how fast the player bullets move. Now read through the code and see if you can work out how change the bullets speed. Try making it faster, then slower. If you're feeling adventurous, try making each bullet move at a random speed. You have all the information you need to do this in previous tutorials. Once done, reset the code to the original value.*

Now let's take a look at **alienstuff.agc**. Again, there is very little here that needs explaination, and although all of the code is documented, we won't repeat our commets where the code very closely resembles previous code. As always, if you are having any problems understanding, read back through previous code snippets and comments. You may find comparing the code here with the code in the **playerstuff.agc** as it is very similar. We will take a closer look the the little function that targets the alien fire as the maths used has a lot of uses and is easy to intergrate into other projects.

```
Function spawnalienship()

//find a free alien
find=0

//flag it if it's not actice and jump out of the loop
For findspare=1 to 20
        If aliens[findspare].active=0
                find=findspare
```

```
                findspare=21
        Endif
Next findspare

//let's make the alien
If find<>0
        //decide which side of the screen it will start from
                aliens[find].direction=RandomSign(1)

                //and set it's X (across position) as approprate.
                If aliens[find].direction=-1
                        aliens[find].x=1080
                Endif

                If aliens[find].direction=1
                        aliens[find].x=-64
                Endif
                        //we will place them at a random position
                        //it's worth noting that we also used a random angle for the alien
                        //as this is used in the alien ships movement.
                        aliens[find].y=random(80,200)
                        aliens[find].angle=random(0,360)
                        SetSpritePosition(aliens[find].sprite,aliens[find].x,aliens[find].y)
                        //set it active
                        aliens[find].active=1
                        //give it a random speed
                        aliens[find].speed=random(2,4)


                //set it on screen
                SetSpritePosition(aliens[find].sprite,aliens[find].x,aliens[find].y)

        Endif

Endfunction

Function movealienships()

        //run through the active aliens
        For movealiens=1 to 20
                If aliens[movealiens].active=1

                //increase the aliens angle by it's movement directon
                inc aliens[movealiens].angle,aliens[movealiens].direction

                //move the aliens in an arc using a little math.
                //We multiple speed  to give it some variation
                xmove#=sin(aliens[movealiens].angle)*aliens[movealiens].speed
                ymove#=cos(aliens[movealiens].angle)*aliens[movealiens].speed

                //update the variable that hold the alien position
                aliens[movealiens].x=aliens[movealiens].x+xmove#*frameadjust
                aliens[movealiens].y=aliens[movealiens].y+ymove#*frameadjust

                //move them left or right depending on direction

aliens[movealiens].x=aliens[movealiens].x+(aliens[movealiens].direction*aliens[movealiens].sp
eed)*frameadjust

                //update the screen position


        SetSpritePosition(aliens[movealiens].sprite,aliens[movealiens].x,aliens[movealiens].y
)

                        //if the player is still alive
                        //there is a small chance the alien will fire at you.
                        If random(1,1000)>998 and player.dead=0
                        //find the free alien bullet
                        usethisone=findfreealienfire()
                        //if one is found create the bullet
                        If usethisone<>0
                        playsound(alienzap)
        setalienfiredest(aliens[movealiens].x+32,aliens[movealiens].y+32,player.x,player.y,us
ethisone)
                        Endif
                        EndIf

                //if the alien has left the screen, make it inactive, and therefore available
for reuse.
```

```
                If aliens[movealiens].direction=-1 and aliens[movealiens].x<-120
                        aliens[movealiens].active=0
                EndIf

                If aliens[movealiens].direction=1 and aliens[movealiens].x>1080
                        aliens[movealiens].active=0
                EndIf


                Endif
        //keep trying
        Next movealiens

EndFunction

Function findfreealienfire()

        //this routime finds a free alien bullet and returns it
        // this is similar to the find a free bullet included
        //in the playerstuff.agc which is part of the makeplayerfire() function
        //we've included this as a separate function to demonstrate how
        //a user function can return a value
        use=0

        //find the free (inactive) alien bullet
        //store it in use and end the loop
        for find=1 to 40
                if afire[find].active=0
                        use=find
                        find=41
                endif
        next find
//return the value from the function
EndFunction use

Function setalienfiredest(x1,y1,x,y,number)

//we pass the 'number' of the alien fire we want to use into the function.

//store the starting cordinates
originalX# = x1
originalY# = y1

  // work out the destination
   destinationX# = x
   destinationY# = y
    distanceX# = destinationX# - originalX#
    distanceY# = destinationY# - originalY#
    //apply a litte bit of math
    distanceFromAtoB# = sqrt ( ( distanceX# * distanceX# ) + ( distanceY# * distanceY# ) )

//if the distance isn't 0 (sitting on top of the target, then adjust the movement
    if ( distanceFromAtoB# <> 0.0 )
        //set the alien bullets position and movement required
        //reach the bullets target position
       afire[number].dx = distanceX# / distanceFromAtoB#
       afire[number].dy = distanceY# / distanceFromAtoB#
       //make the bullet active
       afire[number].active=1
       afire[number].x=x1
       afire[number].y=y1
    endif

 endfunction

 Function movealienfire()

         //this code is very similar to the code to move the player bullets.
         For movefire=1 to 40
                 If afire[movefire].active<>0
                         //instead of firing in a set direction as we do for the
                         //player bullet
                         //we adjust the alien bullet by a pre-calculated amount
                         //we created this in setalienfiredest(x1,y1,x,y,number)
                         //and will move the bullet towards it's target position
                        Inc afire[movefire].x,afire[movefire].dx*6
                        Inc afire[movefire].y,afire[movefire].dy*6

                        //has the bullet exceed 656 down the screen
                        //if so we can assume it's hit the terrain
```

```
                        //and we will reset the bullet and make some dust
                        If afire[movefire].y>656
                                afire[movefire].active=0
                                makebang(afire[movefire].x,afire[movefire].y,1)
                                afire[movefire].x=-10000
                                afire[movefire].y=-10000
                        Endif

                        //has it hit the buggy?
                        //if so, let's make a bang
                        If GetSpriteCollision(afire[movefire].sprite,player.sprite)
                                afire[movefire].active=0
                                makebang(afire[movefire].x,afire[movefire].y,3)
                                explodebuggy()
                                afire[movefire].x=-10000
                                afire[movefire].y=-10000
                        EndIf


        SetSpritePosition(afire[movefire].sprite,afire[movefire].x,afire[movefire].y)

                Endif
         Next movefire

 EndFunction
```

EXERCISE: *At the moment the aliens will fire at the player providing they aren't right on top of them, making quite a difficult game, Read through the code looking for* **setalienfiredest(x1,y1,x,y,number)** and change it so that the aliens will fire from further away. Tip: read the comments!

We will very quickly take a look at the **terrainstuff.agc**, but all of these routines should very familar. Read through the comments for further information and you will see some code you will be very comfortable with now.

```
Function checkterrainhit(movefire)

        //run through the terrain
        For checkhit=1 to 50
        //has the bullet sprite (movefire) hit a rock?
        If terrain[checkhit].active=1 and terrain[checkhit].image=rock
                If GetSpriteCollision(terrain[checkhit].sprite,pfire[movefire].sprite)
                //if so, make the terrain (rock) inactive,
                terrain[checkhit].active=0
                //make an explosion
                makebang(terrain[checkhit].x,terrain[checkhit].y,2)
                //increase the player score
                inc player.score,50
                //move the rock off screen
                SetSpritePosition(terrain[checkhit].sprite,-10000,-10000)
                //make the player bullet inactive
                pfire[movefire].active=0
                //move it offscreen
                SetSpritePosition(pfire[movefire].sprite,-1000,-1000)
                Endif
        EndIf
        //keep checking if needed
Next checkhit

EndFunction

Function drawterrain()

//run through all the terrain pieces
For moveterrain=1 to 50
        //if one is active
        if terrain[moveterrain].active=1
                //move it to the left
                Dec terrain[moveterrain].x,4*frameadjust
                //set it's new position

        SetSpritePosition(terrain[moveterrain].sprite,terrain[moveterrain].x,terrain[moveterr
ain].y)
                //has it left the screen
                If terrain[moveterrain].x<-64
```

```
                //if so make it inactive
                terrain[moveterrain].active=0
                //and make a new piece of terrain off the right hand side of the screen
                maketerrain()
                Endif
        EndIf
        //keep checking
        Next moveterrain

EndFunction

Function maketerrain()

//find a space
space=findspare()

//if there is a free space, which there always should be
If space<>0
        //make it active
        terrain[space].active=1
        //set it off screen to the right
        terrain[space].x=1084
        terrain[space].y=656
        //make it solid ground for now
        terrain[space].image=ground
        //set the image
                SetSpriteImage(terrain[space].sprite,terrain[space].image)

                If random(1,100)>95
                        //5% change we'll change it to a crater
                        terrain[space].image=hole
                        //set the image
                        SetSpriteImage(terrain[space].sprite,terrain[space].image)
                Endif

                If random(1,100)>95 and terrain[space].image=ground
                //further 5% chance of making
                //a rock, but only over solid terrain
                space=findspare()
                terrain[space].active=1
                //rocks appear higher than the terrain
                terrain[space].x=1084
                terrain[space].y=600
                //set the image
                terrain[space].image=rock
                SetSpriteImage(terrain[space].sprite,terrain[space].image)

                Endif

        Endif

EndFunction

Function findspare()

//this rountine works exactly like the findfreealienfire()
//function found in alienstuff.agc
//refer to the comments there if need be
space=0

For search=1 to 50
        if terrain[search].active=0
                space=search
                search=51
        EndIf
Next search

EndFunction space

Function moveterrain()

        //we have a backdrop that move slower
        //in the background
        //take a look at the image in the media folder
        //it is mirrored half way across the image
        //at 1280 pixels in
        //this allow us to appear to have
        //a seemless continious scrolling
        //background by repositioning the image back to 0
        //at the point where there is an exact overlap
```

```
        //move the background
        dec backgroundx,1*frameadjust

        //as mentioned above, reset if the background is at point -1280
        //halfway through the image
        If backgroundx=-1280
                backgroundx=0
        EndIf

        //set the background position to it's new cordinates
        SetSpritePosition(background3,backgroundx,410)

        //reward the player for making progress.
        inc player.score,1

EndFunction
```

EXERCISE: *At the moment the game is very challanging. We can adjust how difficult the game is by reducing or increasing the number of rocks and holes. Take a look at the code and first try changing the chance of rocks appearing and then holes. Make them appear at different intervals, having less chance for an easier game and more chance for a super human challange.*

The last code file we'll be looking at is the explosion code and here we introduce some new elements, firstly sprite animation, and secondly we will have a brief introduction to 2D physics. So, let's take a look at **explosionstuff.agc**.

For simplicities sake, the code below is fully commented, but as some of these commands have muliple uses and can at first glance appear quite complicated we advise reading through the code first, and then selecting each of the physics and animation commands (or any you want more information on) and pressing F1 to read a though a full description of the command and it's syntax.

```
Function explodebuggy()

        //if the player has died (i.e. this function has been called)
        //reduce the number of player lives.
        dec player.lives,1
        //flag the player as dead
        player.dead=1
        //move the player sprite off screen
        SetSpritePosition(player.sprite,-10000,-1000)

        //now we work some illusionary magic
        tempx=player.x
        //we have some broken pieces that look just like
        //the buggy when placed next to each other
        //like a jigsaw
        //Note: we are running this loop backwards, and we use
        //the new command STEP -1 to indicate we will jump backwards.
        //You can sue the STEP command to skip through a FOR - NEXT
        //at any speed and in both directions
                For placedebris=157 to 153 step-1
                //we grab the sprites width for the
                //debris piece
                tempwidth=GetSpriteWidth(placedebris)
                //set it's position
                SetSpritePosition(placedebris,tempx,player.y)
                //then move the sprite position along by it's
                //width, so each sprite is correctly placed
                //next to each other
                Inc tempx,tempwidth
                //finish the loop
        Next placedebris

        //we'll run through each debris sprite
        For bang=150 to 157
                //and turn phyiscs on
                //in this case we are using dynamic physics (2), the most
                //common setting for natural physics objects
                SetSpritePhysicsOn(bang,2)
                //give it some force
```

```
                SetSpritePhysicsAngularImpulse(bang,random(800,1000))
                //if it's a wheel...
                If bang<player.wheel3+1
                //allow it to bounce
                SetSpritePhysicsRestitution(bang,1)
                //and let it rotate
                SetSpritePhysicsCanRotate(bang,1)
                //and of course, give it some force
                SetSpritePhysicsAngularImpulse(bang,random(3500,6000))

                Endif
                //keep going
        Next bang

        //set the time the player died
        //they will be dead for 4000 milliseconds before
        //the game resets,
        //see the main.agc code
        player.deadtime=currenttime

EndFunction

Function makebang(x,y,whichtype)

        //find a free space
        usethis=findfreebang()

        //if it exists
        If usethis<>0

                //set the sprites position
                bangit[usethis].x=x
                bangit[usethis].y=y
                //make it active
                bangit[usethis].active=1

                //run through the different types of explosions
                //1 = dust (when an alien bullet hits the terrain)
                //2 = a flash, for when a rock explodes
                //3 = an explosion for when an alien ship is destroyed.
                Select whichtype

                        //dust
                        Case 1
                                //set the sprite image
                                SetSpriteImage(bangit[usethis].sprite,dust)
                                //set the animation frame
                                //the dust sprite frame is
                                //24 wide, 24 high and has 12 frames.
                                SetSpriteAnimation(bangit[usethis].sprite,24,24,12)
                                //set the sprite playing at 30 frame a second
                                PlaySprite(bangit[usethis].sprite,30)
                                PlaySound(dustsound)
                        EndCase

                        //rock
                        Case 2
                                //set the sprite image
                                SetSpriteImage(bangit[usethis].sprite,rockbang)
                                //set the animation frame
                                //the rock explosion sprite frame is
                                //64 wide, 64 high and has 6 frames.
                                SetSpriteAnimation(bangit[usethis].sprite,64,64,6)
                                //set the sprite playing at 12 frames a second
                                PlaySprite(bangit[usethis].sprite,12)
                                //play the sound for the exploding rock
                                PlaySound(rocksound)
                        EndCase

                        //ship
                        Case 3
                                //set the sprite image
                                SetSpriteImage(bangit[usethis].sprite,shipbang)
                                //set the animation frame
                                //the ship explosion is
                                //64 sprite wide, 70 pixel hight and has 8 frames
                                SetSpriteAnimation(bangit[usethis].sprite,64,70,8)
                                //set the sprite playing at 12 frames a second
                                PlaySprite(bangit[usethis].sprite,12)
                                //play the ship exploding sound
```

```
                        PlaySound(shipsound)
                EndCase

            EndSelect

            //as we are changing the sprites image
            //these images can be smaller or larger than the sprite
            //we can change the sprite size to match the image
            //by using a default width and height of -1. This will
            //set the sprite to have the same width and height as the image.
            SetSpriteSize(bangit[usethis].sprite,-1,-1)
        Endif
EndFunction


Function findfreebang()

        //find a free explosion
        //this is exactly like the code for finding a player fire space
        //and an alien fire space
        use=0

        For find=1 to 40
        if bangit[find].active=0
            use=find
            find=41
        Endif
        Next find

EndFunction use

Function controlbangs()

        //now we will take a look at the explosion sprites
        For check=1 to 40
            //as always, if it active
            If bangit[check].active=1
                //move is slightly to the left to match the lands movement
                bangit[check].x=bangit[check].x-1*frameadjust
                //we can use GetSpriteCurrentFrame to find out the current frame it's
using
                //we can use GetSpriteFrameCount to find out how many frames it has
                //if GetSpriteCurrentFrame()=GetSpriteFrameCount()
                //we know the sprite has played all of it's frames
                If
GetSpriteCurrentFrame(bangit[check].sprite)=GetSpriteFrameCount(bangit[check].sprite)
                    //so we make it inactive (and available again)
                    //and move it off screen completely
                    bangit[check].active=0
                    bangit[check].x=-10000
                    bangit[check].y=-10000
                EndIf
                //update the sprite position

        SetSpritePosition(bangit[check].sprite,bangit[check].x,bangit[check].y)
            EndIf
            //keeep on checking
        Next check

EndFunction
```

EXERCISE: *We've set the sprites physics to something we think works, but why don't you play around with some of the settings.*

- *Try changing the* SetSpritePhysicsAngularImpulse(sprite,impulse) *values to see the difference.*
- *Play around with* SetSpritePhysicsCanRotate(sprite, 1 or 0) *to see the affect this has on sprites when they collide.*
- *Read through the sprite physics commands and see if you can add some interesting effects to your bouncing sprites.*